Лабораторная работа № 2. Первоначальная настройка git

Студент: Чжу Ино
Группа: НПИбд-03-25
Дата: 2026-10-14

---

1. Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

2. Формулировка задания

1. Установить git и gh.
2. Выполнить базовую настройку git.
3. Создать ключи SSH.
4. Создать ключи PGP.
5. Настроить GitHub.
6. Добавить SSH-ключ в GitHub.
7. Настроить gh.
8. Создать репозиторий курса на основе шаблона.

3. Установка программного обеспечения

Выполнена установка git и gh:

```
sudo dnf -y install git
sudo dnf -y install gh
```



Рис. 1: Версия git

```
[zhuyinuo@fedora os-intro]$ gh --version
gh version 2.79.0 (2025-09-09)
```

Рис. 2: Версия gh

4. Базовая настройка git

Задано имя и email владельца репозитория, настроен utf-8, задано имя начальной ветки master, параметры autocrlf и safecrlf:

```
git config --global user.name "     "
git config --global user.email "1132258331@rudn.ru"
git config --global core.quotepath false
git config --global init.defaultBranch master
git config --global core.autocrlf input
git config --global core.safecrlf warn
```

```
[zhuyinuo@fedora os-intro]$ git config --global --list
user.signingkey=B70BAC537B9E029A
user.name=Чжу Ино
user.email=1132258331@rudn.ru
commit.gpgsign=true
gpg.program=/usr/bin/gpg
credential.https://github.com.helper=
credential.https://github.com.helper=!/usr/bin/gh auth git-credential
credential.https://gist.github.com.helper=
credential.https://gist.github.com.helper=!/usr/bin/gh auth git-credential
safe.directory=/media/sf_desktop/test-sign
```

Рис. 3: Глобальная конфигурация git

5. Создание ключей SSH

Сгенерированы ключи RSA (4096 бит) и Ed25519.

Содержимое открытого ключа Ed25519:

6. Создание ключей PGP

Сгенерирован PGP-ключ (RSA 4096).

Экспорт открытого PGP-ключа:

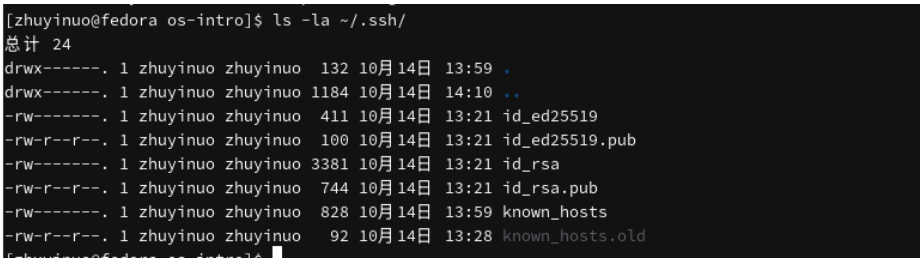


Рис. 4: Список SSH-ключей



Рис. 5: Открытый ключ SSH

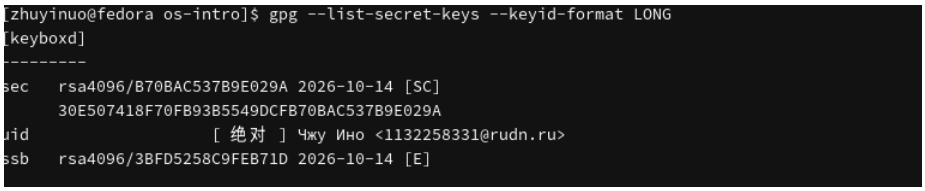


Рис. 6: Список PGP-ключей

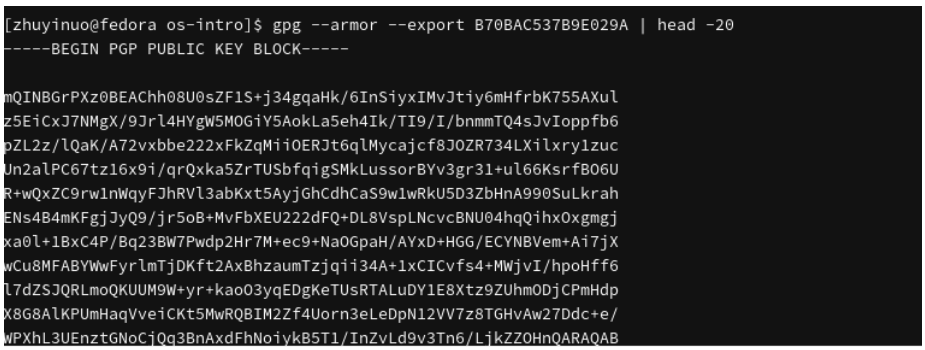


Рис. 7: Открытый PGP-ключ

7. Настройка GitHub

SSH-ключ добавлен в учётную запись GitHub:

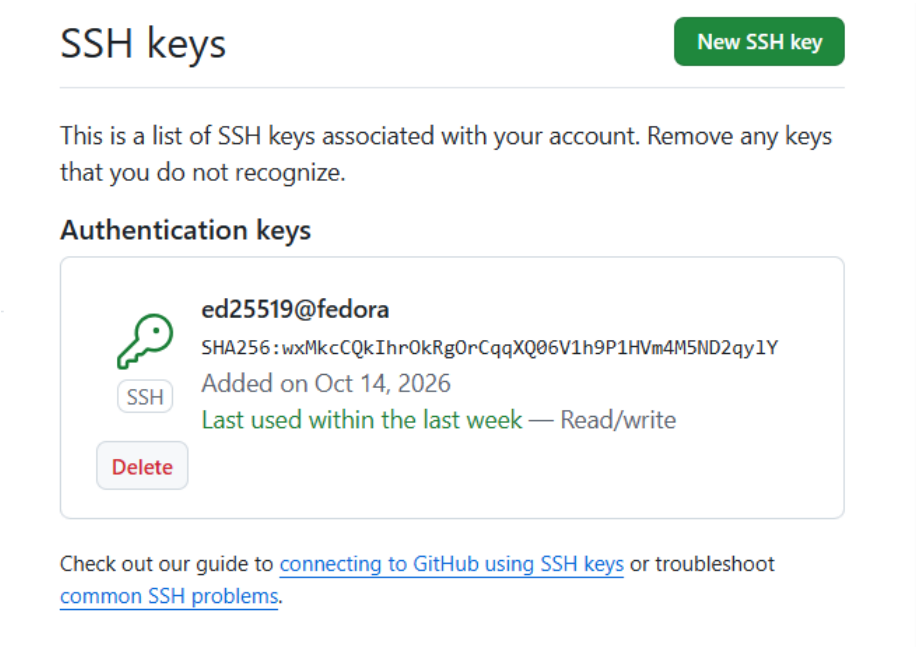


Рис. 8: SSH-ключ на GitHub

PGP-ключ добавлен в учётную запись GitHub:

Настроены автоматические подписи коммитов:

```
git config --global user.signingkey B70BAC537B9E029A
git config --global commit.gpgsign true
git config --global gpg.program $(which gpg)
```

8. Настройка gh

Выполнена авторизация gh:

```
gh auth login
```

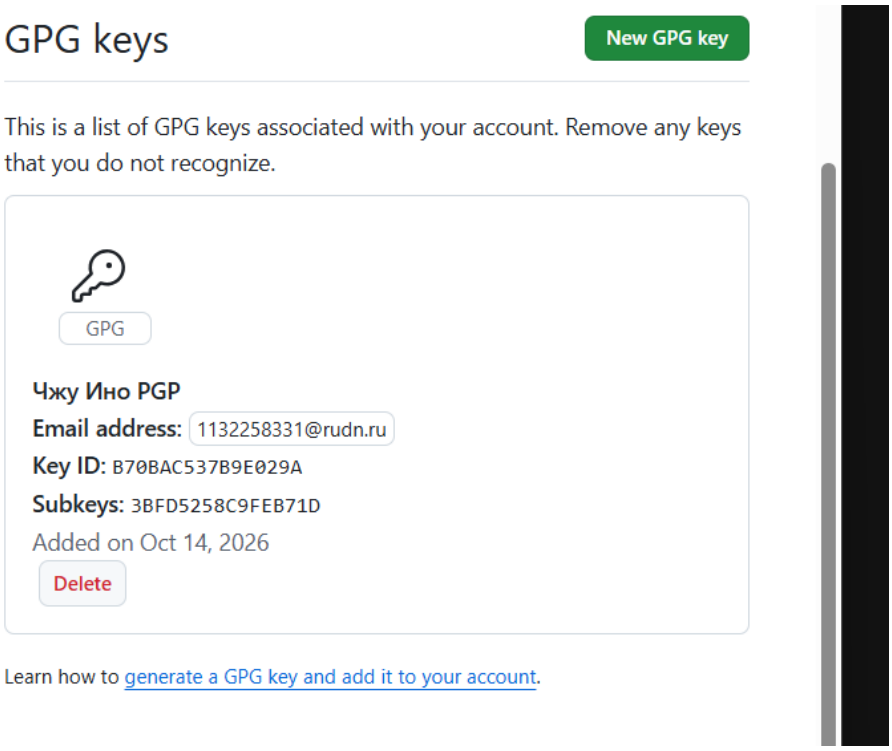


Рис. 9: PGP-ключ на GitHub

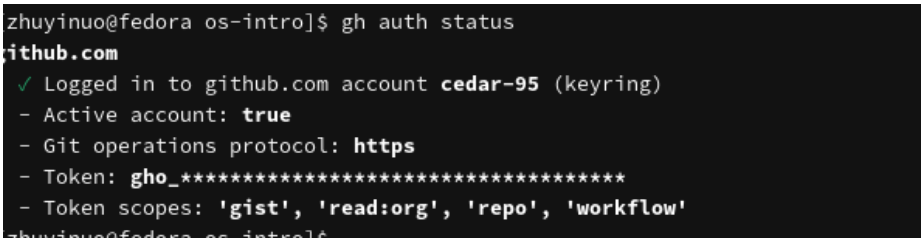


Рис. 10: Статус gh

9. Шаблон для рабочего пространства

9.1. Создание репозитория курса

```
mkdir -p ~/work/study/2026-2/
cd ~/work/study/2026-2/
gh repo create study_2026-2_os-intro --template=yamadharma/course-directory-student-template
git clone --recursive git@github.com:cedar-95/study_2026-2_os-intro.git os-intro
```

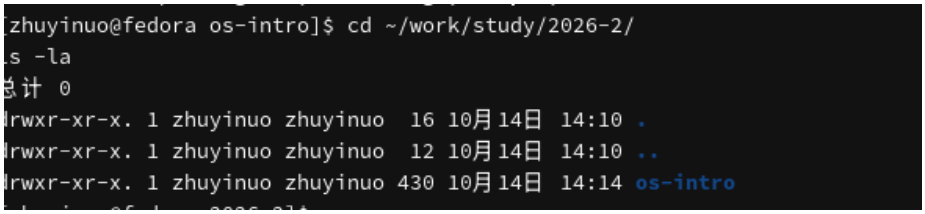


Рис. 11: Клонирование репозитория

9.2. Настройка каталога курса

```
cd ~/work/study/2026-2/os-intro
echo os-intro > COURSE
make prepare
```



Рис. 12: Структура курса

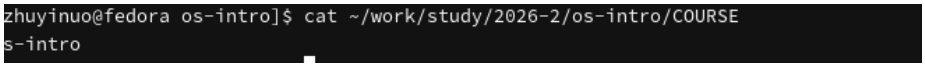


Рис. 13: Файл COURSE

10. Выводы

В ходе лабораторной работы изучена идеология систем контроля версий. Освоены навыки работы с git: базовая настройка, создание ключей SSH и PGP, настройка GitHub, авторизация gh, создание репозитория курса на основе шаблона.



Рис. 14: Подписанный коммит



Рис. 15: История коммитов

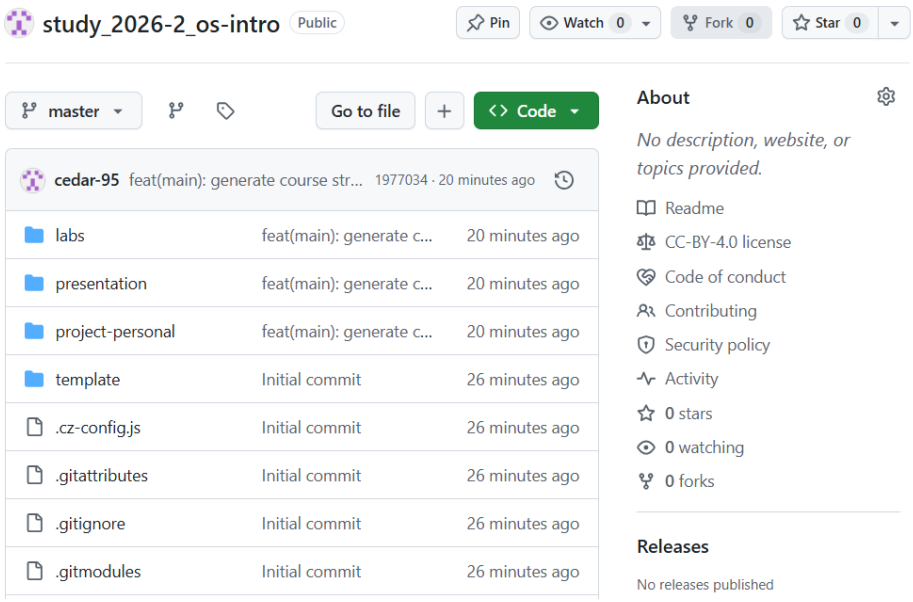


Рис. 16: Репозиторий на GitHub

11. Ответы на контрольные вопросы

1. Что такое системы контроля версий (VCS)?

VCS — системы, позволяющие фиксировать изменения в проекте, совмещать изменения разных участников, откатывать к предыдущим версиям.

2. Объясните понятия VCS: хранилище, commit, история, рабочая копия.

- Хранилище — место хранения всех версий проекта.
- Commit — зафиксированное изменение.
- История — последовательность коммитов.
- Рабочая копия — локальная версия файлов пользователя.

3. Чем отличаются централизованные и децентрализованные VCS?

- Централизованные (CVS, Subversion) — единый сервер.
- Децентрализованные (Git, Bazaar, Mercurial) — у каждого полная копия.

4. Действия с VCS при единоличной работе.

Получение версии → изменения → фиксация → размещение новой версии.

5. Порядок работы с общим хранилищем VCS.

Получение обновлений → работа в локальной ветке → фиксация → отправка.

6. Основные задачи git.

Управление версиями, ветвление, слияние, работа с удалёнными репозиториями.

7. Команды git.

- `git init` — создание репозитория.
- `git add` — добавление в индекс.
- `git commit` — фиксация.
- `git push` — отправка.
- `git pull` — получение.
- `git status` — статус.
- `git diff` — изменения.
- `git branch` — ветки.

- `git checkout` — переключение.
- `git merge` — слияние.

8. Примеры работы с локальным и удалённым репозиториями.

Локальный: `git init`, `git add`, `git commit`. Удалённый: `git remote add`, `git push`, `git pull`.

9. Что такое ветви (branches)?

Ветви — параллельные линии разработки. Позволяют вести работу над несколькими функциями одновременно.

10. Как игнорировать файлы при commit?

Файл `.gitignore` содержит шаблоны игнорируемых файлов.

12. Список литературы

1. Чакон С., Штрауб Б. Git для профессионального программиста. — СПб. : Питер, 2016.
2. Лоэл Д., Маккалоу М. Pro Git. — Apress, 2014.
3. Колисниченко Д. Н. Самоучитель системного администратора Linux. — БХВ-Петербург, 2011.